

Tarea 1: Computación de Alto Rendimiento (IIC3533)

Josefina Abbott Cerda, José Ignacio Salas Coeymans y Borja Smith Vergara

11 de septiembre de 2026

(a) Generación de datos sintéticos

Para generar los datos sintéticos, seguimos los siguientes pasos:

1. Generar los datos, usando el generador aleatorio de NumPy (`np.random.default_rng`) fijando una semilla base, nosotros la establecimos como 1111.
2. Generar el vector de coeficientes reales $\beta^* \in \mathbb{R}^{301}$ muestreando de una distribución normal estándar $\mathcal{N}(0, 1)$.
3. Crear una matriz base de $100\,000 \times 300$ con valores independientes $\mathcal{N}(0, 1)$.
4. Concatenar horizontalmente un vector columna de unos a la matriz base para incorporar el intercepto (β_0), obteniendo así la matriz de diseño $X \in \mathbb{R}^{100\,000 \times 301}$.
5. Calcular el vector de respuesta $y \in \mathbb{R}^{100\,000}$ con el producto matricial $y = X\beta^* + \varepsilon$, donde ε es un vector de ruido con 100 000 muestras i.i.d. de $\mathcal{N}(0, 1)$.

Para correr el script de generación de datos, se puede ejecutar el siguiente comando en la terminal:

```
python src/data_generation.py
```

(b) Implementaciones de bootstrapping

Evolución de `bs_auto.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza la configuración por defecto de scikit-learn. Emplea `LinearRegression()` pese a que X ya contiene una columna de unos; además, al guardar solamente `coef_`, pierde el intercepto separado del modelo. Por ello esta versión sirve como punto de partida, pero no entrega correctamente el intervalo de β_0 .
- **Versión 2:** Introduce `fit_intercept=False`. Dado que la matriz de diseño X ya incluye una columna de unos pre-concatenada, esto evita el cálculo de un intercepto redundante.

- **Versión 3:** Reemplaza el ciclo `for` por una comprensión de listas para expresar de forma más compacta la extracción de coeficientes. Este cambio no vectoriza los ajustes ni garantiza por sí solo una mejora medible.
- **Versión 4:** Declara explícitamente `bootstrap=True` (aunque ya es el valor por defecto) y fija `random_state=1234`, garantizando muestreo con reemplazo y reproducibilidad.
- **Versión 5:** Mantiene los parámetros óptimos del modelo, pero revierte la extracción de coeficientes al ciclo `for` con `.append()` para contrastar el manejo de memoria final.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	Intercepto activo, ciclo <code>for</code>	66.040 s
v2	Sin intercepto, ciclo <code>for</code>	84.483 s
v3	Sin intercepto, list comprehension	118.939 s
v4	Sin inter., reproducible, bootstrap explícito	105.072 s
v5	Sin inter., ciclo <code>for</code> , bootstrap explícito	93.132 s

Cuadro 1: Tiempos de ejecución empíricos para distintas iteraciones de `bs_auto.py`.

3. Análisis de la versión óptima

La **Versión 4** es computacional y estructuralmente la más óptima.

En primer lugar, al definir `fit_intercept=False`, se evita estimar un intercepto separado cuando la matriz X ya incluye la columna de unos. Declarar `bootstrap=True` hace explícito el remuestreo exigido, mientras que `random_state` permite reproducir los mismos intervalos al cambiar el número de procesos.

Los tiempos variaron entre 66 y 119 s y no siguen una tendencia monótona; una sola medición por versión no permite atribuir esas diferencias a la forma de extraer solo 48 vectores. La v1 fue la más rápida, pero es incorrecta para β_0 . Elegimos la v4 por claridad, correctitud y reproducibilidad, no por ser el mínimo aislado. Los valores se guardaron en `results/computer_1/timings_b.csv`.

Evolución de `bs_sklearn.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza el generador aleatorio global (`np.random.choice`) dentro de cada proceso trabajador. Su estado no queda asignado explícitamente a cada remuestreo y, dependiendo de cómo se creen los procesos, puede producir secuencias duplicadas o resultados no reproducibles. Además, mantiene activado el cálculo del intercepto por defecto.
- **Versión 2:** Aísla la generación de números aleatorios asignando una semilla determinista única (`seed_b`) a cada proceso mediante `np.random.default_rng(seed_b)`.

Esto incrementa levemente el tiempo de ejecución por el costo de instanciar 48 generadores distintos, pero es matemáticamente obligatorio para garantizar la validez estadística.

- **Versión 3:** Introduce `fit_intercept=False`. Al igual que en `bs_auto.py`, esto elimina el procesamiento redundante de la matriz, reduciendo los tiempos de ejecución sin cambiar el vector de coeficientes retornado a los procesos trabajadores.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	RNG global (no reproducible), intercepto activo	87.877 s
v2	RNG independiente, intercepto activo	87.005 s
v3	RNG independiente, sin intercepto	71.948 s

Cuadro 2: Tiempos de ejecución empíricos para distintas iteraciones de `bs_sklearn.py`.

3. Análisis de la versión óptima

La **Versión 3** representa la implementación correcta y más eficiente utilizando scikit-learn de forma explícita. El manejo independiente de las semillas hace reproducibles los remuestreos, mientras que la desactivación del intercepto evita trabajo redundante y redujo el tiempo observado a 71.948 s.

Evolución de `bs_numpy.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Traduce literalmente la fórmula de Mínimos Cuadrados Ordinarios entregada en el enunciado: $\hat{\beta} = (X^T X)^{-1} X^T y$. Utiliza `np.linalg.inv()` para calcular explícitamente la matriz inversa antes de realizar las multiplicaciones matriciales.
- **Versión 2:** Reemplaza la inversión explícita reescribiendo el problema como la resolución de un sistema de ecuaciones lineales: $(X^T X)\hat{\beta} = X^T y$. Utiliza `np.linalg.solve()` para encontrar los coeficientes de manera directa.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	Cálculo explícito con <code>np.linalg.inv()</code>	5.480 s
v2	Resolución de sistema con <code>np.linalg.solve()</code>	5.566 s

Cuadro 3: Tiempos de ejecución empíricos para distintas iteraciones de `bs_numpy.py`.

3. Análisis de la versión óptima

La **Versión 2** es la implementación óptima y definitiva para esta sección.

En esta ejecución aislada, `inv` fue 0.086 s más rápido, una diferencia demasiado pequeña para establecer una ventaja. Elegimos `solve` porque evita construir explícitamente la inversa y es la formulación numérica apropiada para resolver el sistema.

Al utilizar `np.linalg.solve()`, NumPy delega la operación a subrutinas optimizadas de álgebra lineal (LAPACK), que resuelven el sistema $(X^\top X)\hat{\beta} = X^\top y$ sin construir explícitamente la inversa. Esto reduce operaciones y error numérico respecto de `inv`; no obstante, ambas versiones forman las ecuaciones normales y, por tanto, siguen siendo menos estables que resolver directamente el problema de mínimos cuadrados mediante QR o SVD.

Para correr las implementaciones, se pueden ejecutar los siguientes comandos en la terminal:

```
python src/bs_auto.py
python src/bs_sklearn.py
python src/bs_numpy.py
```

(c) Correctitud y reproducibilidad

Primero calculamos $\hat{\beta}$ sobre el conjunto completo, como indica el paso 1 del algoritmo. El error relativo $\|\hat{\beta} - \beta^*\|_2 / \|\beta^*\|_2$ fue 0.00311, lo que confirma que el ajuste recupera adecuadamente los coeficientes verdaderos.

Luego ejecutamos cada versión dos veces con $p = 1$ y una vez con $p = 4$, usando las mismas semillas. La Tabla 4 resume la cobertura, el ancho medio de los intervalos y la máxima diferencia entre ejecuciones.

Implementación	Cobertura	Ancho medio	Rep. $p = 1$	$p = 1$ vs. $p = 4$
BaggingRegressor	91.69 %	0.011282	0	0
sklearn + joblib	93.36 %	0.011373	0	0
NumPy + joblib	93.36 %	0.011373	0	0

Cuadro 4: Correctitud y reproducibilidad. Las dos últimas columnas muestran la máxima diferencia absoluta entre extremos de los intervalos.

Las coberturas son similares y razonablemente cercanas al 95 % nominal, considerando que solo se usan $B = 48$ remuestreos. Las versiones manuales emplean exactamente los mismos índices y sus intervalos difieren como máximo en $4,80 \times 10^{-14}$, debido al redondeo de las rutinas de álgebra lineal. **BaggingRegressor** usa otra secuencia de remuestreo y por eso no produce los mismos extremos, pero su cobertura y ancho son comparables.

La reproducibilidad requiere fijar la semilla de los datos, asignar una semilla determinista a cada remuestreo y fijar `random_state` en **BaggingRegressor**. Como la semilla depende de b y no del worker que ejecuta la tarea, cambiar p no cambia los intervalos. Los resultados completos se almacenan en `results/computer_1/correctness_c.csv`.

Para verificar la correctitud y reproducibilidad de las implementaciones, se puede ejecutar el siguiente comando en la terminal:

```
python src/correctness.py
```

(d) Backend multiprocessing de joblib

Creación de procesos y asignación de memoria: `Parallel` utiliza por defecto el backend basado en procesos `loky`. Este crea intérpretes Python separados, cada uno con su propio espacio de direcciones; por tanto, no debe asumirse que todos los objetos se comparten automáticamente mediante *fork*. En sistemas que crean procesos con *fork* puede existir inicialmente *Copy-on-Write*, pero `loky` también serializa argumentos para enviarlos a los workers y su comportamiento depende del sistema operativo.

Comportamiento de los arreglos X e y : Como X supera el umbral de tamaño por defecto de joblib, `loky` puede volcarlo automáticamente a un archivo temporal y abrirlo como *memmap* de solo lectura en los workers. Esto evita serializar una copia completa de X por tarea. El arreglo y es menor y puede serializarse normalmente. Sin embargo, la indexación avanzada $X[\text{indices}]$ y $y[\text{indices}]$ sí crea arreglos nuevos $X^{(b)}$ e $y^{(b)}$ dentro de cada tarea. Por tanto, se comparte o mapea la entrada original, pero cada worker necesita memoria adicional para sus remuestreos y matrices intermedias.

(e) Actividad del sistema y oversubscription

Los experimentos de este inciso se realizaron en un MacBook Air con Apple M1, 8 cores (4 de rendimiento y 4 de eficiencia), 8 GB de RAM y macOS (arquitectura ARM64). Se utilizó NumPy 2.0.2, joblib 1.5.3 y `threadpoolctl` 3.6.0. La configuración de NumPy indica que las operaciones BLAS y LAPACK están enlazadas con el framework Apple Accelerate.

Ejecutamos `bs_numpy.py` tres veces con $p \in \{1, 2, 4, 8\}$. Se observaron exactamente 1, 2, 4 y 8 procesos trabajadores, respectivamente. En una ejecución prolongada con $p = 8$, los workers registraron aproximadamente entre 53 % y 66 % de CPU cada uno, confirmando que los remuestreos se distribuyen entre procesos independientes.

p	Workers observados	Tiempo mediano [s]
1	1	5.052
2	2	4.409
4	4	4.003
8	8	5.531

Cuadro 5: Actividad observada al ejecutar la versión NumPy.

Ejecutamos `threadpool_info()` dentro de cada worker, después de cargar NumPy. En todos los casos retornó una lista vacía. Esta observación es coherente con que el NumPy instalado usa Apple Accelerate, cuyo pool interno no es reportado por `threadpoolctl`. Al cargar SciPy y scikit-learn sí aparecen un pool OpenBLAS y uno OpenMP de 8 threads, pero estos pertenecen a dichas bibliotecas y no demuestran que `np.linalg.solve` use esos pools.

El tiempo mejora hasta $p = 4$, pero empeora un 38 % al pasar de $p = 4$ a $p = 8$. Esto evidencia saturación y competencia por memoria; también es compatible con *oversubscription*, aunque la falta de visibilidad sobre Accelerate impide conocer el valor efectivo de t y atribuirle por sí sola la degradación.

En general, sí existiría oversubscription si cada uno de los p procesos lanzara t threads BLAS y se cumpliera $pt > p_{\text{máx}}$. En una instalación compatible con `threadpoolctl` puede corregirse envolviendo el cálculo de cada worker con `threadpool_limits(limits=1)` o configurando `inner_max_num_threads=1` en el backend `loky`. Para Apple Accelerate puede fijarse `VECLIB_MAXIMUM_THREADS=1` antes de iniciar Python. Las observaciones completas quedaron almacenadas en `results/computer_1/observations_e.csv`.

Anexo (e): repetición en el computador 2 con OpenBLAS

Como se explicó arriba, con el NumPy instalado mediante `pip` en macOS el backend BLAS es Apple Accelerate y `threadpool_info()` devuelve una lista vacía, por lo que en ninguno de los dos computadores fue posible observar ni limitar los threads internos que pide este inciso. Para poder responder esa parte del enunciado, en el computador 2 se repitió el experimento en un ambiente conda (miniforge, canal conda-forge), que es la configuración recomendada por el enunciado. En ese ambiente NumPy queda enlazado a OpenBLAS, que `threadpoolctl` sí reconoce. Se mantuvieron las mismas versiones (Python 3.9, NumPy 2.0.2, scikit-learn 1.6.1, joblib 1.5.3, threadpoolctl 3.6.0), los mismos scripts y los mismos datos; la única diferencia respecto del experimento original es la librería BLAS. Este anexo no entra en la comparación del inciso (j) porque el computador 1 no lo ejecutó. Los archivos están en `results/computer_2_openblas/`.

p	Workers	Threads OpenBLAS por worker	Threads totales	Tiempo mediano [s]
1	1	10	10	4.368
2	2	5	10	4.212
4	4	2	8	3.321
8	8	1	8	2.714
10	10	1	10	2.517

Cuadro 6: Anexo (e), computador 2 con OpenBLAS: `threadpool_info()` dentro de cada worker sí reporta el pool y su número de threads. Sin límite, cada proceso abre 10 threads.

La Tabla 6 muestra lo que antes no se podía ver: cada worker reporta un pool OpenBLAS y, sin que nosotros fijemos ningún límite, el número de threads por worker es 10, 5, 2, 1 y 1 para $p = 1, 2, 4, 8, 10$. Esto no es casualidad: el backend `loky` de `joblib` fija por defecto en cada worker las variables de entorno `OPENBLAS_NUM_THREADS`, `OMP_NUM_THREADS`, `MKL_NUM_THREADS` y `VECLIB_MAXIMUM_THREADS` al valor $\text{máx}(\lfloor p_{\text{máx}}/p \rfloor, 1)$, de modo que el total de threads BLAS sea aproximadamente $p_{\text{máx}}$ para cualquier p . Es decir, `joblib` evita la oversubscription por diseño, y lo hace tanto para OpenBLAS como para Accelerate (la variable `VECLIB_MAXIMUM_THREADS` es la que Accelerate obedece).

Esto permite reinterpretar las observaciones hechas con Accelerate. En el computador 2, durante el inciso (f) se observaron procesos al ~ 280 % de CPU con $p = 3$, lo que coincide con $\lfloor 10/3 \rfloor = 3$ threads por worker; en el computador 1, los workers al 53–66 % de CPU con $p = 8$ coinciden con $\lfloor 8/8 \rfloor = 1$ thread cada uno. Con $p = 1$ `joblib` no crea workers y el cálculo corre en el proceso principal, donde Accelerate usa todos los núcleos. En consecuencia, en el

experimento original el valor efectivo de t no era desconocido sino $\lfloor p_{\text{máx}}/p \rfloor$, y la degradación observada en el computador 1 al pasar de $p = 4$ a $p = 8$ no se explica por oversubscription de threads, sino por la presión de memoria de mantener ocho remuestras de ~ 241 MB en 8 GB de RAM.

Para mostrar qué ocurre cuando la oversubscription sí existe, se forzó el límite dentro de cada tarea con `threadpool_limits(limits=t)` en combinaciones con $pt > p_{\text{máx}}$, lo que anula la protección de joblib.

(p, t)	Threads totales pt	Mediana [s]	Veces más lento que la mejor
(9, 1)	9	2.644	1.0
(2, 10)	20	37.493	14.2
(4, 5)	20	18.018	6.8
(5, 10)	50	125.618	47.5
(10, 5)	50	57.247	21.7
(10, 10)	100	132.847	50.3

Cuadro 7: Oversubscription provocada en el computador 2 con OpenBLAS: combinaciones con $pt > 10$ frente a la mejor combinación válida.

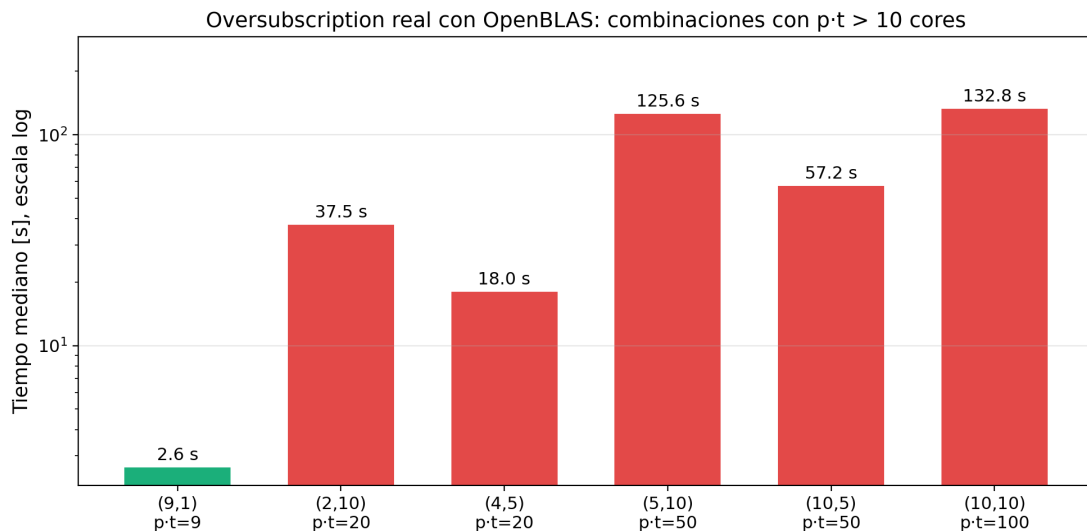


Figura 1: Oversubscription provocada en el computador 2 con OpenBLAS. Eje vertical en escala logarítmica.

El efecto es severo: con 20 threads sobre 10 núcleos el tiempo se multiplica por 7 a 14, y con 50 o 100 threads por 22 a 50. Con 100 threads (10 procesos con 10 threads cada uno) el bootstrap tardó 132.8 s frente a 2.6 s de la mejor combinación válida. La causa es que los threads de OpenBLAS con capa OpenMP esperan activamente (*busy-wait*) a sus compañeros; cuando hay muchos más threads que núcleos, la mayor parte del tiempo de CPU se gasta en esperas y cambios de contexto, no en aritmética. La corrección es justamente la que aplica

joblib por defecto: mantener $pt \leq p_{\text{máx}}$, ya sea con `inner_max_num_threads` en `Parallel`, con `threadpool_limits` dentro de la tarea o con las variables de entorno antes de iniciar Python.

(f) Tiempos de ejecución en función de p

El primer computador posee $p_{\text{máx}} = 8$ cores lógicos. Comparamos las versiones finales `bootstrap_auto_v4`, `bootstrap_sklearn_v3` y `bootstrap_numpy_v2` usando los mismos datos ($N = 100\,000$, $k = 300$), $B = 48$ y las mismas semillas. Para cada implementación y cada $p \in \{1, \dots, 8\}$ se realizaron tres ejecuciones, cada una en un intérprete Python nuevo para impedir que joblib reutilizara un pool creado por una medición anterior. El orden de las 72 configuraciones fue aleatorizado para reducir el sesgo causado por calentamiento del equipo o por el orden de ejecución. Reportamos la mediana, por ser menos sensible que la media a interferencias transitorias del sistema.

p	BaggingRegressor	sklearn + joblib	NumPy + joblib
1	86.843	84.737	6.017
2	56.377	60.699	5.029
3	50.298	55.099	4.232
4	56.254	55.999	5.497
5	46.518	59.245	4.357
6	62.251	72.303	5.479
7	64.378	114.517	5.039
8	73.594	127.574	4.288

Cuadro 8: Mediana de los tiempos de ejecución $T(p)$, en segundos, para tres repeticiones aisladas en el primer computador.

La implementación basada solamente en NumPy fue la más rápida para todos los valores de p : con un proceso tardó aproximadamente 6.017 s, frente a 86.843 s de `BaggingRegressor` y 84.737 s de la versión manual con scikit-learn. La diferencia se debe principalmente a que `LinearRegression` realiza más trabajo general para ajustar cada modelo, mientras que la versión NumPy resuelve directamente las ecuaciones normales.

Las dos implementaciones basadas en scikit-learn mejoraron al aumentar p hasta un punto intermedio. El mejor tiempo mediano fue 46.518 s con $p = 5$ para `BaggingRegressor` y 55.099 s con $p = 3$ para `sklearn + joblib`. Con $p = 8$, scikit-learn empeoró hasta 127.574 s, debido a la presión de memoria de los remuestreos de 100 000 filas en un equipo con 8 GB. NumPy también obtuvo beneficio paralelo: su mejor tiempo fue 4.232 s con $p = 3$, aunque $p = 5$ y $p = 8$ entregaron tiempos cercanos. La mejora está lejos de ser lineal por el overhead, las copias de cada remuestra y la competencia por ancho de banda de memoria.

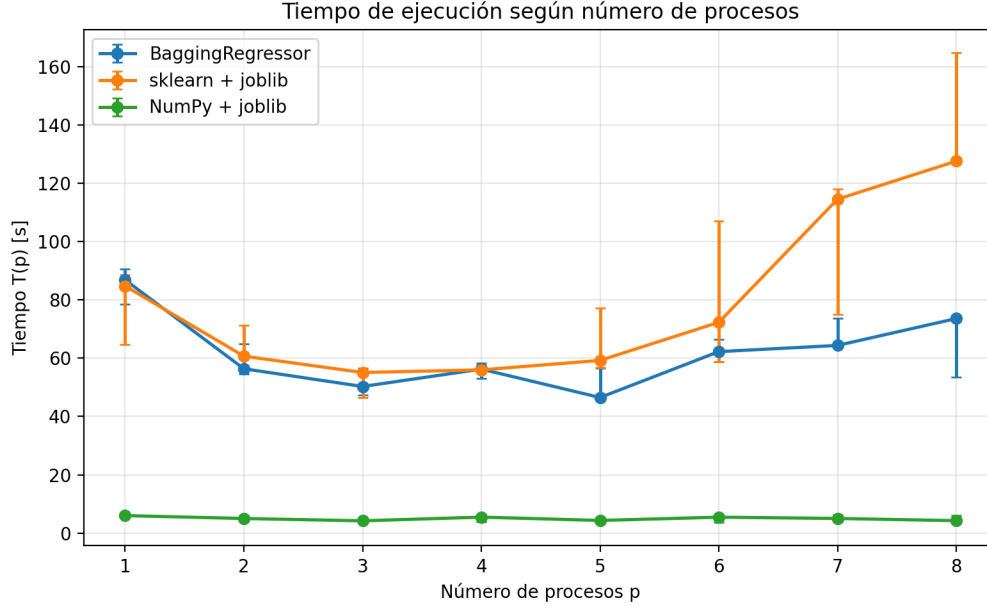


Figura 2: Tiempo mediano de ejecución en función de p . Las barras se extienden entre el mínimo y el máximo de las tres repeticiones.

(g) Speedup $S(p)$ y Eficiencia $E(p)$

A partir de las medianas de la Tabla 8, calculamos para cada implementación:

$$S(p) = \frac{T(1)}{T(p)}, \quad E(p) = \frac{S(p)}{p}.$$

Elegimos como $T(1)$ la mediana de las tres ejecuciones de la misma implementación con $p = 1$: 86.843 s para **BaggingRegressor**, 84.737 s para **sklearn + joblib** y 6.017 s para **NumPy + joblib**. Esta elección mantiene constantes el algoritmo, los datos, las semillas y la instrumentación, por lo que mide escalabilidad fuerte sin mezclar el resultado con una implementación secuencial diferente. Además, usar la mediana evita que una ejecución excepcionalmente lenta o rápida determine toda la curva.

p	BaggingRegressor		sklearn + joblib		NumPy + joblib	
	$S(p)$	$E(p)$	$S(p)$	$E(p)$	$S(p)$	$E(p)$
1	1.000	1.000	1.000	1.000	1.000	1.000
2	1.540	0.770	1.396	0.698	1.196	0.598
3	1.727	0.576	1.538	0.513	1.422	0.474
4	1.544	0.386	1.513	0.378	1.095	0.274
5	1.867	0.373	1.430	0.286	1.381	0.276
6	1.395	0.233	1.172	0.195	1.098	0.183
7	1.349	0.193	0.740	0.106	1.194	0.171
8	1.180	0.148	0.664	0.083	1.403	0.175

Cuadro 9: Speedup y eficiencia calculados a partir de los tiempos medianos.

Ninguna implementación alcanzó la curva ideal. **BaggingRegressor** obtuvo su mayor speedup en $p = 5$, con $S(5) = 1,867$ y $E(5) = 0,373$. La versión manual con scikit-learn alcanzó

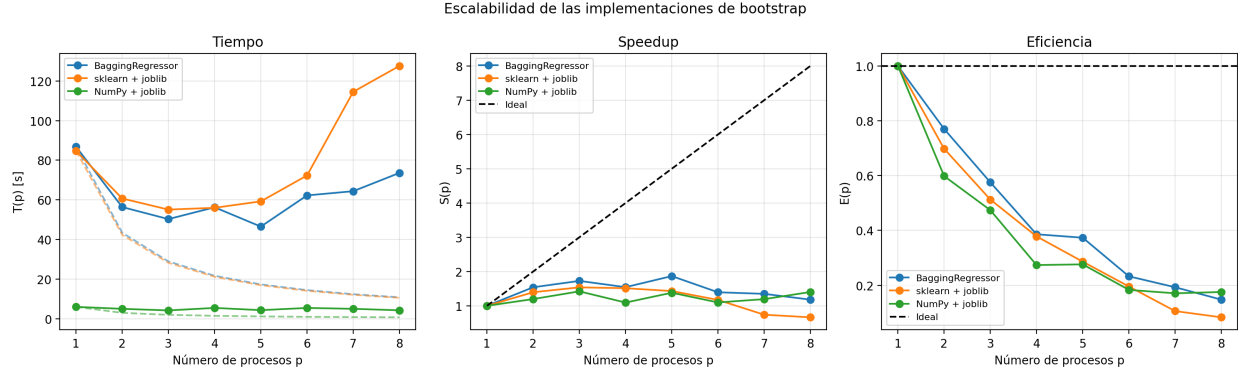


Figura 3: Tiempo, speedup y eficiencia observados. En el panel de tiempo, las líneas discontinuas corresponden a $T(1)/p$ para cada implementación; en los otros paneles, la curva ideal es $S(p) = p$ y $E(p) = 1$.

su máximo en $p = 3$, con $S(3) = 1,538$ y $E(3) = 0,513$. En ambas versiones el trabajo de ajustar 48 regresiones es suficiente para amortizar parte del costo de los procesos, pero la eficiencia disminuye al crecer p debido a creación y coordinación de procesos, distribución de tareas, movimiento de resultados y competencia por memoria.

La versión NumPy es la más rápida en tiempo absoluto y, con el problema más grande, obtiene speedup para todos los valores $p > 1$. Su máximo es $S(3) = 1,422$, con $E(3) = 0,474$. El speedup vuelve a 1.403 en $p = 8$, pero con una eficiencia de solo 0.175; por ello $p = 3$ es preferible: alcanza el menor tiempo usando menos procesos para $B = 48$, $N = 100\,000$ y $k = 300$.

(h) Overhead $T_o(p)$

Definimos el overhead total como

$$T_o(p) = pT(p) - T(1),$$

que mide el tiempo-procesador adicional respecto de la ejecución con un solo proceso. Por definición, $T_o(1) = 0$.

El overhead crece en general con p y explica la separación respecto de la curva ideal. Sus fuentes principales son el arranque de intérpretes, la serialización o mapeo de X e y , el despacho de las 48 tareas, la creación de $X^{(b)}$ e $y^{(b)}$, la recolección de coeficientes, la sincronización y la competencia por memoria y caché. En NumPy el overhead absoluto es menor, pero es grande respecto de su $T(1)$; por eso no obtiene speedup. Las pequeñas irregularidades menores corresponden a variabilidad experimental. El gran aumento de scikit-learn en $p = 7$ y $p = 8$ es consistente con la competencia por memoria al mantener varias remuestras de aproximadamente 241 MB.

p	BaggingRegressor	sklearn + joblib	NumPy + joblib
1	0.000	0.000	0.000
2	25.910	36.662	4.041
3	64.050	80.560	6.679
4	138.173	139.259	15.970
5	145.748	211.488	15.768
6	286.661	349.083	26.855
7	363.803	716.886	29.256
8	501.910	935.855	28.287

Cuadro 10: Overhead total $T_o(p)$, en segundos.

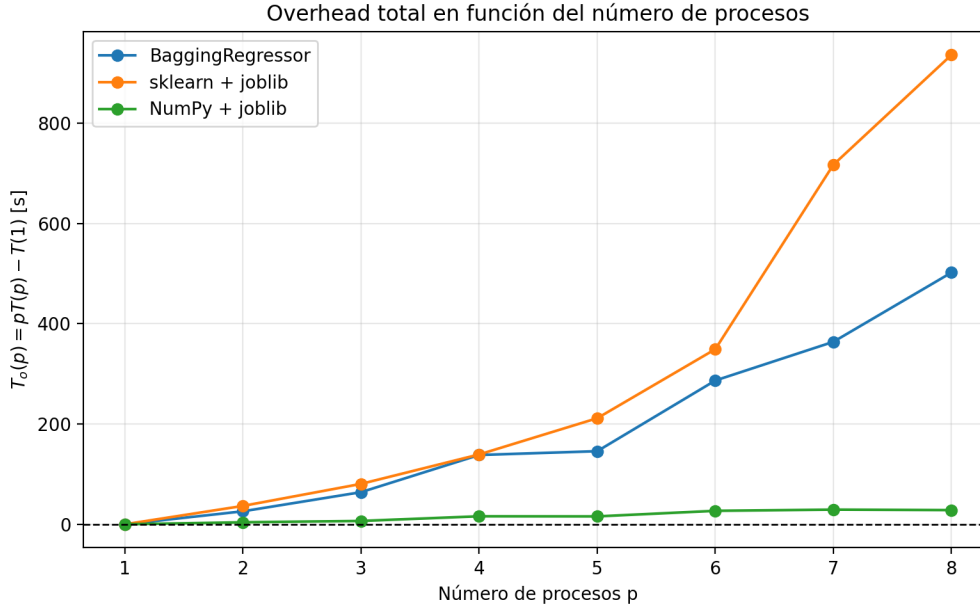


Figura 4: Overhead total en función del número de procesos.

(i) Variación de procesos y threads internos

Usamos la versión NumPy, que fue la más rápida en (b) y (f), y evaluamos las 20 combinaciones enteras (p, t) que satisfacen $pt \leq 8$. El límite se aplicó dentro de cada tarea mediante `threadpool_limits(limits=t)`. Cada combinación se ejecutó tres veces en un intérprete nuevo y se reportó la mediana.

La menor mediana observada fue 4.068 s con $(p, t) = (3, 2)$, seguida por 4.234 s con $(4, 1)$ y 4.436 s con $(3, 1)$. La configuración $(1, 1)$ tardó 5.580 s, por lo que el aumento de trabajo por remuestreo permite ahora amortizar el overhead de varios procesos. En cambio, $(8, 1)$ tardó 5.878 s y no mejora la ejecución secuencial de manera robusta.

Existe una limitación importante: `threadpool_info()` no detecta Apple Accelerate, por lo que t es el límite solicitado, pero no fue posible verificar el número efectivo de threads internos. En consecuencia, la mejor combinación nominal es $(3, 2)$, pero no es posible atribuir

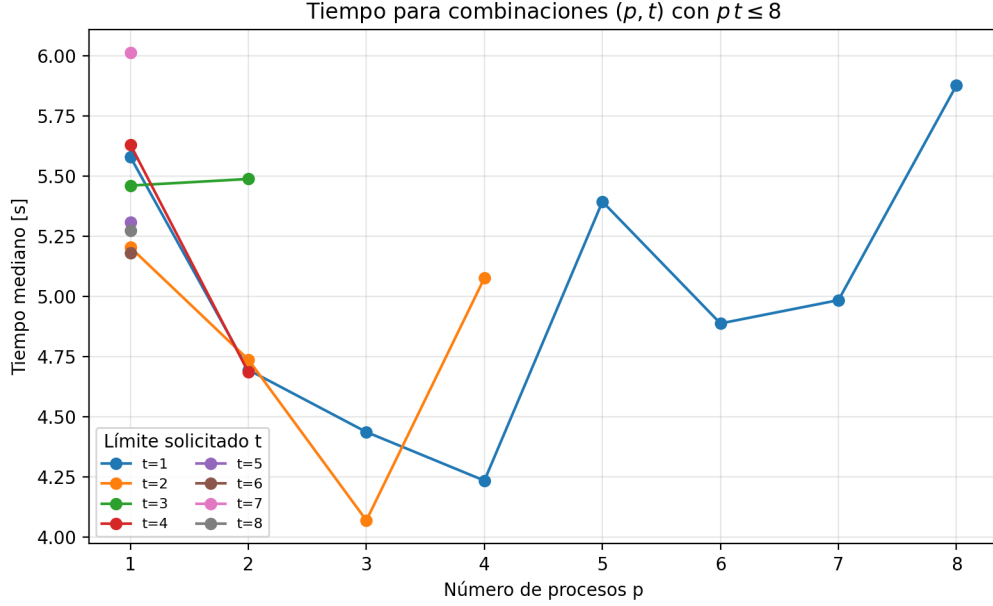


Figura 5: Tiempo mediano para todas las combinaciones (p, t) con $pt \leq 8$.

separadamente la mejora a dos threads internos efectivos. Aun así, esta combinación satisface $pt = 6 \leq 8$, obtiene la menor mediana y evita la saturación observada al usar ocho procesos; por ello es la recomendada para este computador. Los 60 tiempos individuales están en `results/computer_1/timings_i.csv`.

Anexo (i): control efectivo de t en el computador 2 con OpenBLAS

Con el mismo ambiente conda del anexo de (e), se repitieron las 27 combinaciones (p, t) con $pt \leq 10$ del computador 2, cinco veces cada una en un intérprete nuevo. Aquí `threadpool_limits(limits=)` sí actúa sobre OpenBLAS, por lo que t pasa de ser un valor solicitado a un valor efectivo, verificable con `threadpool_info()` en cada corrida.

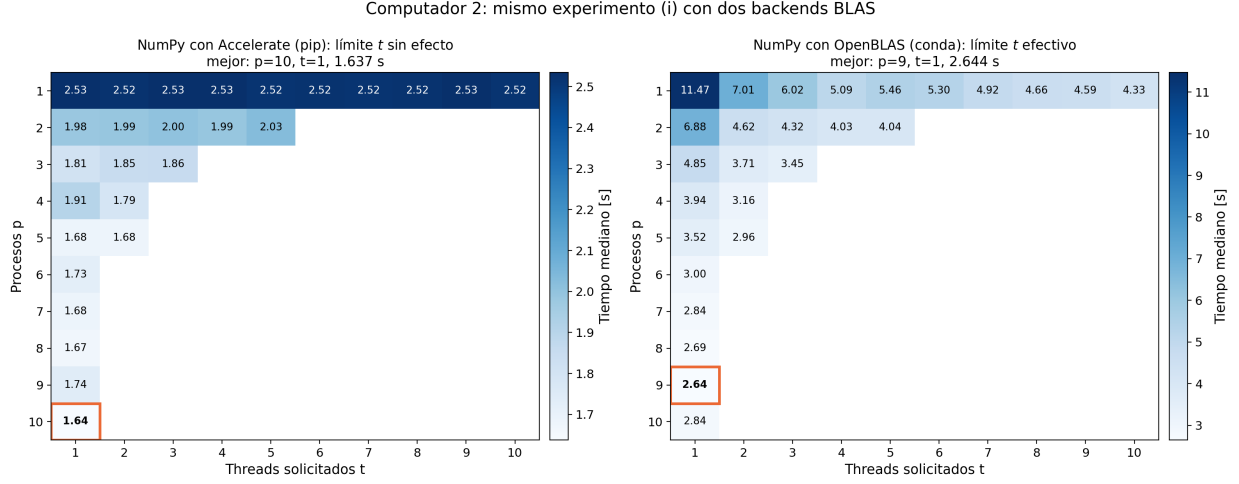


Figura 6: Inciso (i) en el computador 2 con ambos backends. A la izquierda, el experimento original con Accelerate; a la derecha, la repetición con OpenBLAS. La celda destacada es la mejor combinación de cada caso.

(p, t)	Accelerate [s]	OpenBLAS [s]
(1, 1)	2.533	11.470
(1, 2)	2.520	7.009
(1, 3)	2.532	6.020
(1, 4)	2.534	5.095
(1, 5)	2.520	5.458
(1, 6)	2.521	5.300
(1, 7)	2.519	4.918
(1, 8)	2.522	4.663
(1, 9)	2.532	4.591
(1, 10)	2.518	4.328

Cuadro 11: Fila $p = 1$ del inciso (i) en el computador 2 con ambos backends. Con Accelerate el tiempo no depende de t ; con OpenBLAS sí.

La diferencia entre ambos mapas es inmediata en la fila $p = 1$ (Tabla 11). Con Accelerate los diez valores están entre 2.52 y 2.53 s: el límite no llegó a la librería y el proceso principal usó siempre todos los núcleos. Con OpenBLAS la misma fila va de 11.47 s con $t = 1$ a 4.33 s con $t = 10$, es decir, ahora sí se observa el efecto de los threads internos, y además se dispone por primera vez de una medición realmente secuencial: (1, 1) con 11.47 s.

(p, t)	pt	Mediana [s]	Mín [s]	Máx [s]
(9, 1)	9	2.644	2.624	2.716
(8, 1)	8	2.686	2.620	2.692
(10, 1)	10	2.844	2.482	2.866
(7, 1)	7	2.844	2.800	2.886
(5, 2)	10	2.955	2.892	3.122

Cuadro 12: Anexo (i), computador 2 con OpenBLAS: cinco mejores combinaciones (5 repeticiones cada una).

La mejor combinación fue (9, 1) con 2.644 s, seguida muy de cerca por (8, 1) con 2.686 s y por (10, 1) y (7, 1) con 2.844 s; las diferencias entre ellas están dentro de la dispersión de las repeticiones. El patrón general es claro: para un mismo presupuesto de núcleos, repartir el trabajo en procesos rinde más que repartirlo en threads BLAS. Con los mismos 10 núcleos, (1, 10) tardó 4.33 s y (10, 1) 2.84 s; con 8 núcleos, (1, 8) tardó 4.66 s, (2, 4) 4.03 s, (4, 2) 3.16 s y (8, 1) 2.69 s. La razón es que el paralelismo entre procesos es de grano grueso y sin sincronización (cada remuestreo es independiente), mientras que los threads BLAS deben sincronizarse dentro de cada operación matricial y solo paralelizan la parte del cálculo que es álgebra lineal densa; el remuestreo, la indexación y la copia de $X^{(b)}$ quedan en un solo thread.

Este anexo también obliga a releer el inciso (g). Los speedups medidos allí nunca superaron 2 en ninguno de los dos computadores. La explicación principal es que el $T(1)$ usado como referencia no era una ejecución secuencial: con $p = 1$ el cálculo corre en el proceso principal y la librería BLAS usa todos los núcleos disponibles (10 threads en el computador 2, según la Tabla 6). Al aumentar p , joblib reduce los threads por worker a $\lfloor p_{\text{máx}}/p \rfloor$, de modo que el paralelismo total se mantiene cerca de $p_{\text{máx}}$ para todo p ; lo único que cambia es cuánto de ese paralelismo va en procesos y cuánto en threads. Por eso las curvas $T(p)$ son casi planas y $S(p)$ satura pronto. Medido contra la referencia realmente secuencial de este anexo, la mejor combinación alcanza un speedup de $11.47/2.64 \approx 4.3$ con 9 procesos, que es una imagen mucho más fiel de lo que el paralelismo aporta en este problema.

Finalmente, los tiempos absolutos de los dos backends no son comparables entre sí: la mejor combinación con Accelerate (1.64 s) es más rápida que la mejor con OpenBLAS (2.64 s) porque Accelerate aprovecha las unidades de multiplicación matricial del chip Apple. Lo que este anexo aporta no es un tiempo menor, sino la posibilidad de observar y controlar t , que era lo que el enunciado pedía y el experimento original no podía entregar.

(j) Comparación entre computadores

Para este inciso de comparación, se repitieron exactamente los mismos experimentos (con los mismos scripts) de los incisos f, g, h e i, con los mismos ambientes de Python y datos del problema, véase $N = 100\,000$, $k = 300$, $B = 48$ y también la misma semilla para los valores aleatorios, de modo que la comparativa refleje las diferencias en hardware y un análisis de computación. Algo importante a destacar es que se decidió que el computador 1 usara 3

repeticiones por configuración, a diferencia del computador 2, que usó 5; en ambos se usa la mediana. Esta decisión es parte de la argumentación del contraste de computadores.

Inicialmente, en términos de hardware, la primera diferencia observada es que el computador 2 posee más núcleos o cores (10 vs 8) y sus respectivas divisiones entre cores lógicos y de eficiencia, destacando que ambos tienen la misma cantidad de cores de rendimiento; muy importante también mencionar que en los algoritmos cambia el p (cantidad máxima de procesos). Luego se aprecia la diferencia de memoria RAM, siendo 24 GB el computador 2 contra 8 GB, y finalmente la generación de chips, M5 vs M1. Dentro del software de Mac se tiene la “misma” lógica; veremos que los comportamientos de los *schedulers* y librerías internas juegan un papel importante.

Característica	Computador 1	Computador 2
Chip	Apple M1	Apple M5
Cores de rendimiento / eficiencia	4 / 4	4 / 6
Cores lógicos (totales)	8	10
RAM	8 GB	24 GB
Sistema operativo	macOS 14.7.8 (arm64)	macOS 26.6.2 (arm64)
Backend BLAS	Apple Accelerate	Apple Accelerate
Repeticiones por configuración	3	5
Combinaciones (p, t) en (i)	20	27

Cuadro 13: Comparativa de hardware.

Tiempos de ejecución (f)

La primera comparación es sobre el inciso f, la cual compara los tiempos de ejecución entre los 3 algoritmos. La primera comparativa que puede surgir, y que es clave, es ver cómo se comportan ambos computadores utilizando solo 1 proceso, es decir, ver granularmente la diferencia de rendimiento de la unidad básica (core), y es que el análisis parte de acá. Se puede observar que con solo un proceso el computador 2 es significativamente más rápido, reduciendo el tiempo del primer algoritmo en cerca de un 65 %, en el siguiente un 65 % y finalmente un 58 %, es decir, siendo casi 2.9 y 2.4 veces más rápido. Esto ya nos da una pista de la diferencia intrínseca entre un M5 y un M1 sin paralelización.

p	BaggingRegressor		sklearn + joblib		NumPy + joblib	
	C1	C2	C1	C2	C1	C2
1	86.843	30.321	84.737	29.145	6.017	2.484
2	56.377	22.168	60.699	22.877	5.029	1.972
3	50.298	19.133	55.099	19.400	4.232	1.932
4	56.254	16.898	55.999	16.563	5.497	1.861
5	46.518	17.754	59.245	17.506	4.357	1.683
6	62.251	16.451	72.303	16.562	5.479	1.614
7	64.378	15.983	114.517	16.138	5.039	1.666
8	73.594	16.083	127.574	16.074	4.288	1.639
9	–	16.565	–	16.637	–	1.654
10	–	16.223	–	16.439	–	1.590

Cuadro 14: Tiempo mediano en segundos.

Luego de eso, dentro de la comparativa posible (8 procesos), se ve que el computador 2 siempre es mejor que el 1, como se puede observar en la línea naranja; además, no solo es menor sino que más estable, lo que puede significar que el primer computador sufrió problemas de variabilidad durante los experimentos que el segundo computador pudo normalizar. Cabe destacar que en el computador 2 nunca empeora en tiempo al aumentar procesos mientras que en el 1 si, esta diferencia es probablemente debido al uso eficiente de memoria y como veremos más adelante, un overhead menor por cada proceso agregado.

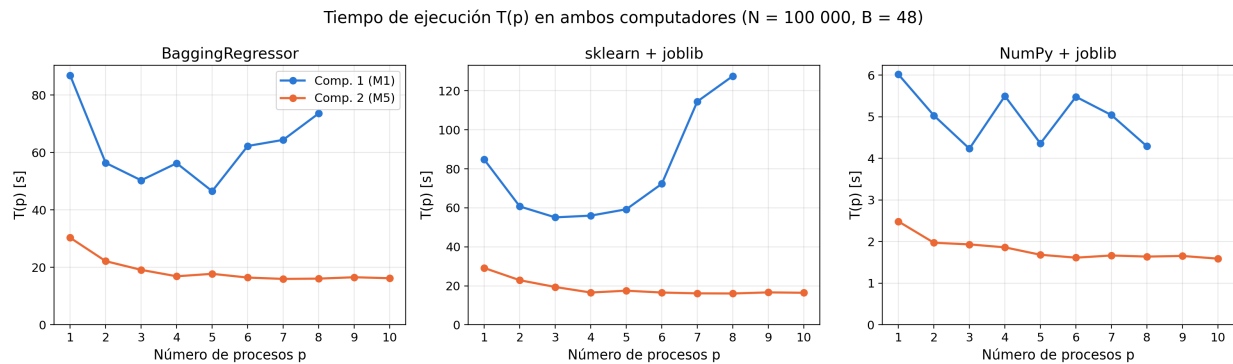


Figura 7: Comparación gráfica de tiempo mediano en segundos.

Speedup y eficiencia (g)

En este inciso se compara qué tanto aumenta la velocidad al aumentar a p los procesos. Al hablar de eficiencia podemos notar que la diferencia de procesadores (M5 y M1) inicialmente no afecta a la métrica, ya que son comparativas con un valor ideal: para $S(p)$ es $T(1)$ y para $E(p)$ es $S(p)$, lo que depende de cada computador de forma independiente.

Como vimos en la comparativa anterior, fijándonos en tiempos brutos, véase $T(1)$, el computador 2 es evidentemente mejor; sin embargo, lo interesante viene cuando comienza el análisis del speedup y eficiencia, ya que, como mencionamos, son métricas relativas: no es qué tan rápido es, sino cuánto más rápido. Debido a lo explicado, podemos ver que los valores de speedup máximos son muy similares.

Implementación	Computador 1 (M1)					Computador 2 (M5)				
	$T(1)$	$S_{\text{máx}}$	E en $S_{\text{máx}}$	$S(8)$	$T_o(8)$	$T(1)$	$S_{\text{máx}}$	E en $S_{\text{máx}}$	$S(8)$	$T_o(8)$
BaggingRegressor	86.84	1.867 ($p = 5$)	0.373	1.180	501.9	30.32	1.897 ($p = 7$)	0.271	1.885	98.3
sklearn + joblib	84.74	1.538 ($p = 3$)	0.513	0.664	935.9	29.15	1.813 ($p = 8$)	0.227	1.813	99.4
NumPy + joblib	6.02	1.422 ($p = 3$)	0.474	1.403	28.3	2.48	1.563 ($p = 10$)	0.156	1.515	10.6

Cuadro 15: Resumen de speedup máximo, eficiencia en el máximo y valores con 8 procesos en cada computador.

El análisis es más interesante cuando se ven las comparativas en el tiempo de ambos computadores. Para el speedup, volvemos a ver que son relativamente similares, con el computador 2 siendo un poco mejor, en gran medida gracias a la estabilidad. En ambos computadores se observa de igual forma que a medida que aumenta p no se acerca a la curva ideal; en el

computador 2 se observa que se mantiene más asintótico. Mientras que para la eficiencia los comportamientos son evidenciamente más similares, con una curva que cae de casi la misma manera. Estos comportamientos tan similares, considerando las diferencias de hardware, nos dan a entender que el software (elemento en común) pone límites que no son solucionables con mejoras de los elementos del computador, sino que atacan a la lógica algorítmica y al código.

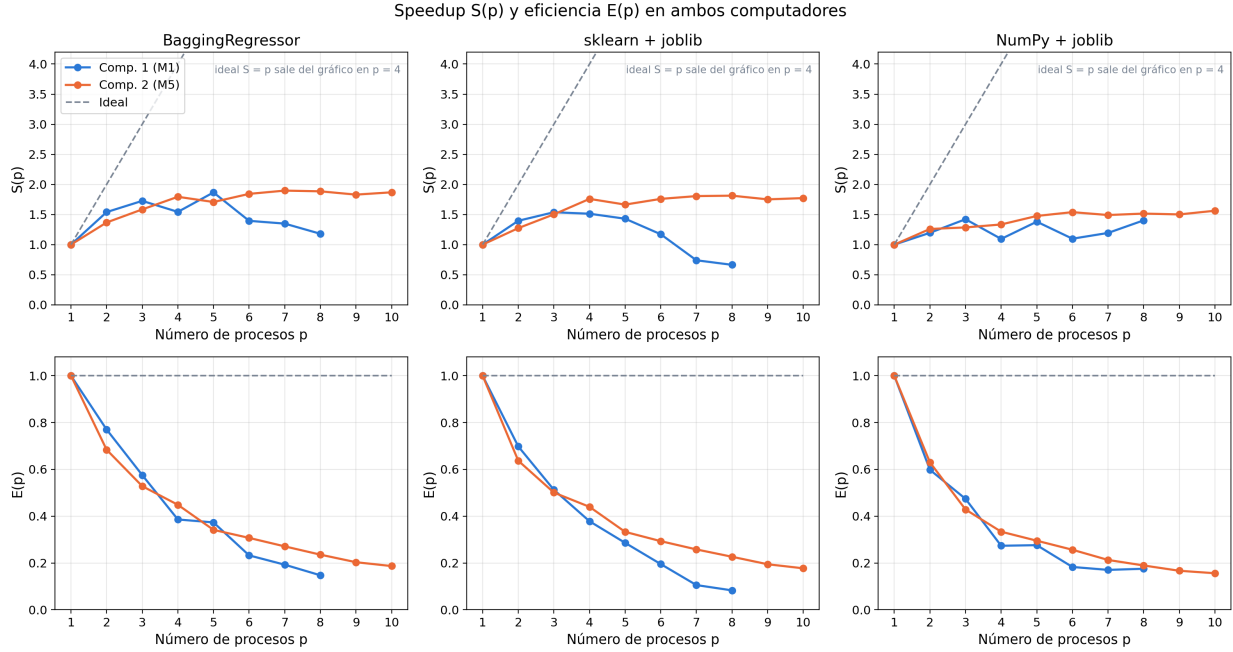


Figura 8: Speedup $S(p)$ (arriba) y eficiencia $E(p)$ (abajo) en ambos computadores.

Overhead (h)

Esta sección compara el inciso h, el cual calcula el tiempo empleado en lo que no es directamente la resolución del algoritmo, véase el overhead, que se calcula como se mencionó en el mismo inciso. En este caso la primera noción es ver que en los gráficos la línea del computador 1 es mucho más pronunciada que la del computador 2. Esto se interpreta como que el computador 1 acumula mucho más overhead al añadir un proceso que el computador 2, lo cual además se ve casi exponencial, mientras que para el computador 2 es más lineal. Si bien, al ser ambos Mac, hay procesos que siguen siendo iguales, la diferencia de memoria juega un rol fundamental: probablemente el computador 1 tiene mucho overhead de cores esperando memoria que ya está colapsada, mientras que el computador 2 tiene más holgura. De hecho, un análisis interesante es que el computador 1 empieza a tener comportamientos erráticos a partir de $p = 4$, donde se acaban los cores de rendimiento (rápidos) y se empieza a delegar a los de eficiencia (más lentos), lo que nos hace pensar que los cores de eficiencia del 1 son más lentos que los del computador 2 (recordemos que tienen la misma cantidad de cores rápidos).

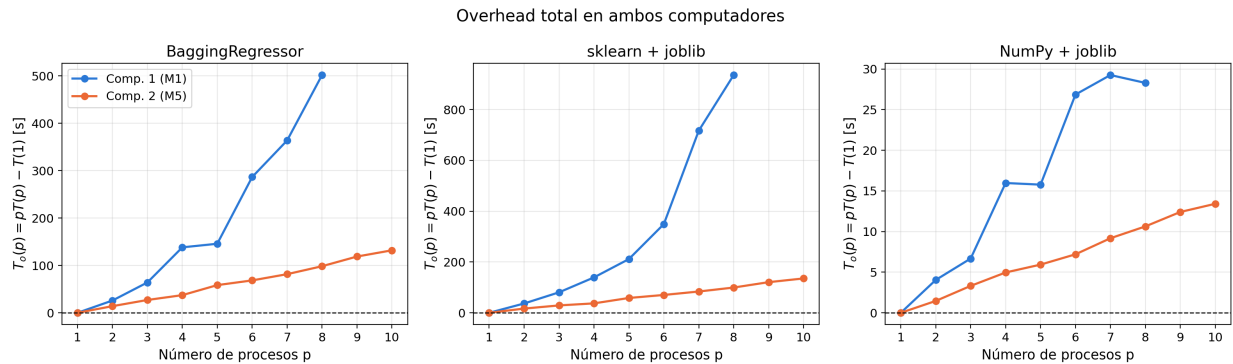


Figura 9: Overhead total $T_o(p) = pT(p) - T(1)$ en ambos computadores.

Combinación de procesos y threads (i)

El análisis comparativo de esta sección es un poco complejo: debido a la naturaleza de los Mac, NumPy utiliza Accelerate como librería para calcular operaciones matriciales en Apple, y threadpoolctl no detecta nativamente esto, por lo que no era verificable el número de threads efectivos internos. Sin embargo, el experimento sí nos dio conclusiones interesantes para analizar.

Lo mencionado anteriormente es visible en que en cada fila los valores son muy similares entre sí, cuando lógicamente no deberían. Esto se debe a que el límite de threads que le pedimos al programa nunca llegó a Accelerate, threadpoolctl solo sabe controlar librerías como OpenBLAS o MKL, y como no ve a Accelerate, la orden de usa solo t threads se aplica a una lista vacía, es decir, a nadie. Accelerate sigue usando la cantidad de threads que decide por su cuenta sin importar el t que pidamos, por lo que el t de nuestras tablas es solo el valor solicitado y no el efectivo, por eso hay filas que no siguen tendencias claras. La decisión de mantener el script así fue por razones comparativas, se realizó un nuevo análisis de la mejor combinación con un script que utiliza un NumPy diferente, sin embargo ese no es utilizable para la comparación.

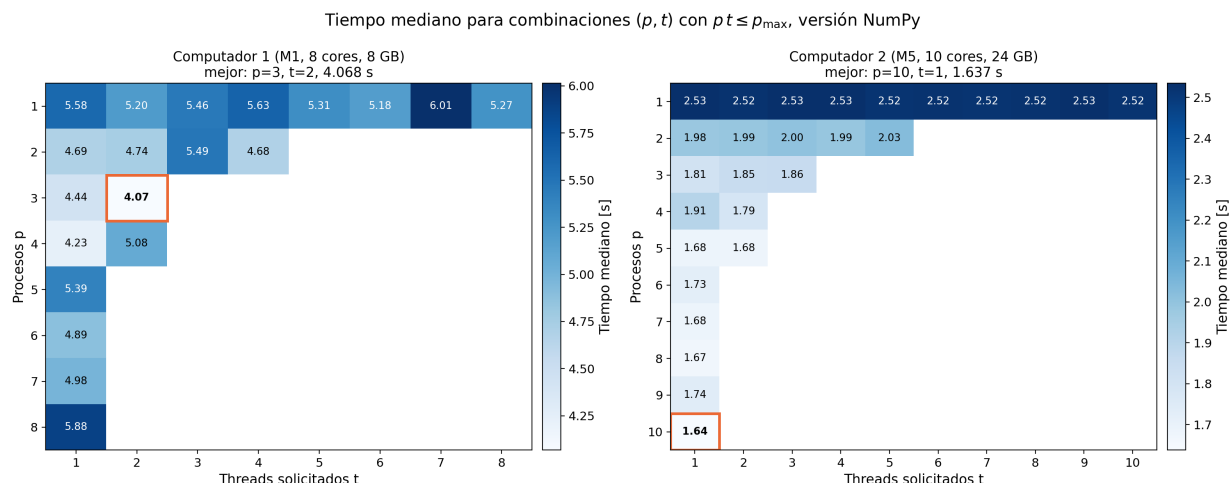


Figura 10: Tiempo mediano de la versión NumPy para cada combinación; en naranja, el más rápido.

Síntesis

Estos experimentos nos ayudaron a llegar a múltiples conclusiones, comenzando desde la demostración de la diferencia entre 2 computadores con hardware diferente: cómo las generaciones de computadores (particularmente en una comparativa de la misma marca) han avanzado y mejorado a través de los años, y cómo la arquitectura de los chips, los aumentos de memoria y la cantidad de cores lógicos afectan directamente a velocidades y rendimiento en computación. A su vez, concluimos que el software es también un factor limitante: por más que se mejore el computador, hay límites lógicos muy interesantes de analizar, donde la velocidad ya no radica en la potencia sino en la lógica de los algoritmos.

Declaración de uso de Inteligencia Artificial

Se utilizó OpenAI Codex y claude como apoyo, particularmente en: Redacción de código en python (generación y debug), formulación de tablas y elementos Latex para el informe, sin embargo todo lo escrito y el análisis del informe fue escrito y validado por los integrantes del grupo, y no se utilizó para generar ideas ni análisis de los resultados.